SPACE FACULTY

Let's learn about **satellites**!

Space mission and **satellite** design course

Satellite Command & Data Handling

2NDSpace

Why is data management crucial in space?

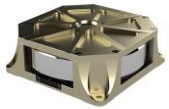
Mission failures due to data systems problems:

- Mars Climate Orbiter (1999) – Lost due to a unit conversion error (imperial vs metric).
- Mars Pathfinder (1997) – Experienced repeated system resets caused by a software priority inversion issue.
- Galileo (Jupiter mission, 1995) – Suffered major data rate limitations after the high-gain antenna failed to deploy.
- Chandrayaan-2 (2019) – The lander was lost during descent due to guidance and control anomalies.
- Galileo (2000s) – Faced operational issues related to the navigation architecture.

SPACE FACULTY
2NDSpace

Satellite Data System Overview

- Communication systems (Transceivers, antennas)
- Power management and distribution (PMAD)
- Electrical power systems (EPS)
- Sensors (Temperature, accelerometers, etc.)
- Reaction wheels and magnetorquers
- Thermal control systems
- Navigation systems (e.g., GPS)
- Payload instruments (Mission-specific)



SPACE FACULTY
2NDSpace

OBDH & OBC

On Board Data Handling (OBDH)

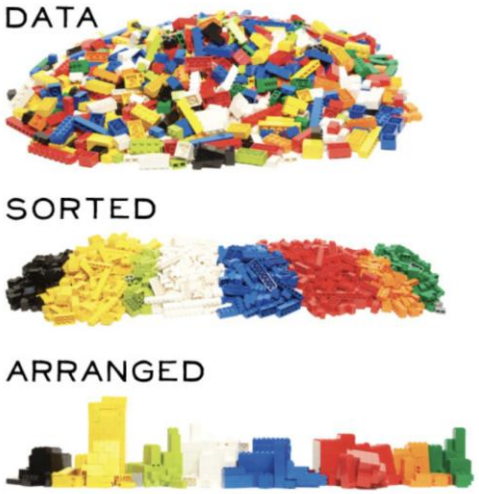
- Data management unit: collects, formats, stores, and routes telemetry and payload data.
- Interfaces-centric: handles packetization, time-tagging, mass memory, and downlink preparation.
- Typically more deterministic: relies on predefined data handling rules and communication protocols.

On Board Computer (OBC)

- Main computing unit: runs flight software and executes mission logic.
- Control-centric: supports guidance/navigation/control, commanding, autonomy, and fault response.
- More "decision-driven": adapts behavior based on sensor inputs and changing mission conditions.

SPACE FACULTY
2NDSpace

OBDH: The Librarian



The OBDH can be compared to the librarian in a library. The librarian is responsible for organizing, cataloging, and managing books and information in the library.

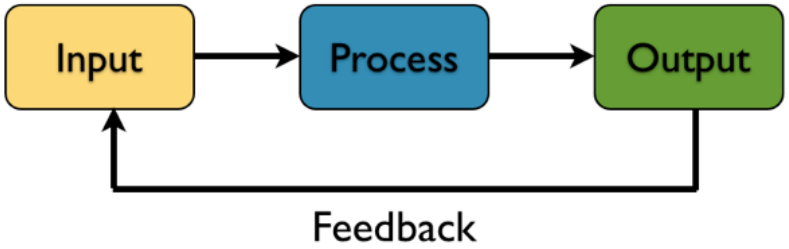
Role in Spacecraft: OBDH focuses on the reception, storage, and management of data generated by various spacecraft systems and sensors. It ensures that data is organized, stored efficiently, and prepared for transmission to ground control.

SPACE FACULTY
2NDSpace

OBC: The Researchers

The OBC is like a team of researchers in the library. Researchers use the library's resources to conduct experiments, analyze data, and make decisions based on their findings.

Role in Spacecraft: The OBC Utilizing the data organized by OBDH. It processes data, executes software, and makes decisions to control the spacecraft's behavior, navigate, and accomplish mission tasks.

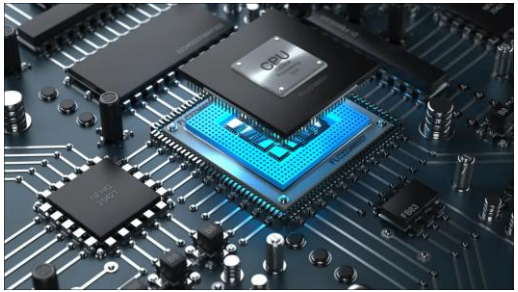


SPACE FACULTY
2NDSpace

OBDH and OBC Interplay

The collaboration between the librarian and the researchers represents the interplay between OBDH and OBC. The librarian ensures that information is readily available and well-organized, while the researchers use that information to achieve specific goals.

Role in Spacecraft: In a similar way, OBDH and OBC work together to ensure the spacecraft's success. OBDH manages data so that OBC can use it effectively for spacecraft control, navigation, and mission execution.



SPACE FACULTY
2NDSpace

Dimensional Aspects

- OBC/OBDH boards must fit the CubeSat standard, respecting keep-outs, harness exits, and access to connectors.
- Interfaces: limited volume forces short, clean cable routing and connector choice.
- Memory and storage: non-volatile storage and any mass memory boards must fit the stack while meeting radiation/retention needs and thermal limits.
- Compute vs. power: higher processing capability usually means higher dissipation; the enclosure and mounting must provide a thermal path otherwise the “best” CPU is unusable.
- Accommodation for payload compute (if any): if the payload needs a payload-OBC, you need extra volume, thermal path, and high-speed links which directly impacts the bus architecture.

SPACE FACULTY
2NDSpace

Main Solutions for OBCs

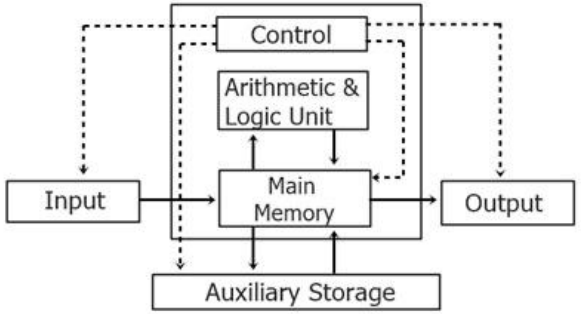
Single-Board Computers (SBCs): These are compact, integrated computers with processors, memory, and I/O interfaces on a single board. SBCs are commonly used in spacecraft for their simplicity and reliability.

Microcontrollers: Microcontrollers are low-power, embedded systems used for specific tasks like sensor interfacing and basic control functions. They are often employed in CubeSats.

Field-Programmable Gate Arrays (FPGAs): FPGAs offer flexibility in hardware acceleration and real-time processing. They are used for tasks requiring high-speed data processing and custom logic.

SPACE FACULTY
2NDSpace

Single-Board Computers



Block Diagram of Computer

Typical Architecture

Central Processing Unit (CPU):

- Integrated CPU with an Arithmetic Logic Unit (ALU) for data processing.
- Executes software instructions and algorithms for mission-critical tasks.

Memory:

- Includes Random Access Memory (RAM) for temporary data storage.
- Utilizes Flash memory or Solid-State Drives (SSDs) for data and program storage.

Input/Output (I/O) Interfaces:

- Interfaces for communication with sensors, instruments, and subsystems.
- Facilitates data exchange and command execution.

Operating System (OS):

- Runs specialized embedded operating systems for software execution.
- Provides a platform for mission-specific software applications.

Software and Algorithms:

- Hosts software applications and algorithms tailored to the mission's needs.
- Manages data processing, control, and communication tasks.

SPACE FACULTY
2NDSpace

Microcontrollers

Typical Architecture

Microcontroller Unit (MCU):

- Contains an integrated CPU core with a simplified Arithmetic Logic Unit (ALU).
- Designed for low-power and specific embedded tasks.

Memory:

- Embedded Flash memory for program storage.
- RAM for temporary data storage during operations.

Input/Output (I/O) Peripherals:

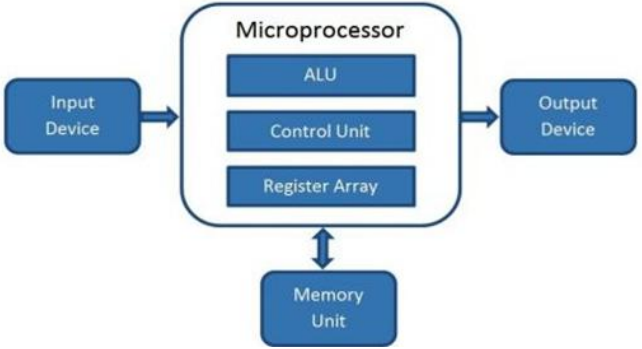
- Interfaces with sensors, actuators, and external components.
- Collects sensor data and manages control functions.

Analog-to-Digital Converter (ADC):

- Converts analog sensor data into digital form for processing.
- Essential for interfacing with analog sensors.

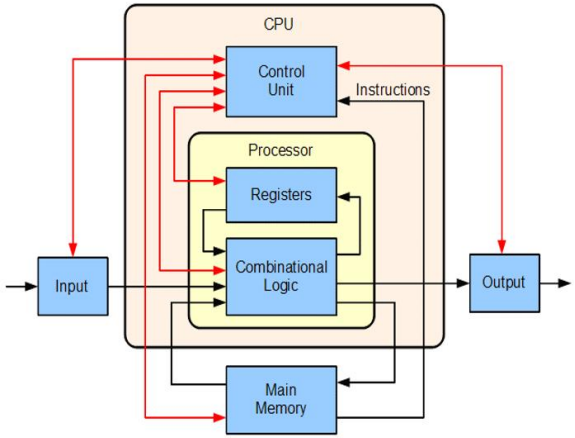
Timers and Counters:

- Used for timing and scheduling tasks, making microcontrollers suitable for real-time applications.



SPACE FACULTY
2NDSpace

FPGAs



Typical Architecture

Configurable Logic Blocks:

- An array of reconfigurable logic blocks that can be programmed to implement custom digital logic circuits.
- Facilitates the creation of specialized hardware.

Input/Output (I/O) Blocks:

- Provides interfaces for external connections, enabling data flow in and out of the FPGA.
- Supports communication with sensors and subsystems.

Interconnect Resources:

- Programmable interconnects that route signals between CLBs and I/O blocks.
- Enables flexible connectivity between logic elements.

Memory:

- Stores the bitstream configuration file, defining the custom logic implemented in the FPGA.
- Allows for reprogramming as needed.

SPACE FACULTY
2NDSpace

Comparing SBCs, Microcontrollers and FPGAs

Similarities:

- Processing Capabilities: All three execute instructions, run software, and process data.
- Memory: Each has storage components for data and programs.
- Input/Output (I/O): Equipped with interfaces for external communication.
- Customization: Configurable for mission-specific tasks.

Differences:

- Complexity: SBCs: Versatile, run diverse software. Microcontrollers: Simpler, for specific tasks. FPGAs: Customizable hardware logic.
- Processing Speed: SBCs: High processing speeds. Microcontrollers: Lower processing speeds, real-time. FPGAs: Real-time, logic-dependent.
- Power Consumption: SBCs: Higher power consumption. Microcontrollers: Low-power for extended missions. FPGAs: Variable based on configuration.
- Reconfigurability: SBCs: Software updates. Microcontrollers and FPGAs: Reprogrammable.
- Use Cases: SBCs: General computing, complex tasks. Microcontrollers: Low-power, sensor interfacing. FPGAs: Custom hardware, signal processing.

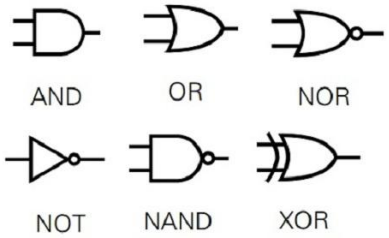
SPACE FACULTY
2NDSpace

Comparing SBCs, Microcontrollers and FPGAs

Another fundamental difference is their **computational approach**. FPGAs rely on Boolean algebra to create custom logic circuits, whereas SBCs and microcontrollers primarily utilize binary arithmetic within software applications.

BOOLEAN

Boolean algebra is a mathematical system used to analyze and manipulate binary variables (0 and 1) using logical operations (AND, OR, NOT, XOR). Boolean algebra deals with logical operations that evaluate the truth values of statements, making it suitable for expressing and simplifying logical conditions.



BINARY

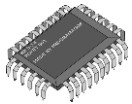
Binary is a base-2 numeral system that represents numbers using only two symbols, 0 and 1. Binary arithmetic involves mathematical operations (addition, subtraction, multiplication, division) performed with binary numbers.

SPACE FACULTY
2NDSpace

Programming Languages

Three main programming languages:

- Python
- C and C++
- VHDL and Verilog (for FPGAs only)



SPACE FACULTY
2NDSpace

Python

Python is a versatile and readable programming language, well-suited for tasks like data analysis, automation, and scripting.

Pros:

- Versatile and readable.
- Rapid development and prototyping.
- Extensive libraries and frameworks.

Cons:

- Slower execution compared to low-level languages.
- May require more memory.

Hardware compatibility:

- Python is used in a variety of development boards and microcontrollers, including Raspberry Pi, Arduino, and more.

SPACE FACULTY
2NDSpace

C and C++

C and C++ are programming languages known for their exceptional speed and efficiency, making them suitable for low-level operations, real-time control, memory management, and balancing performance and code organization.

Pros:

- Exceptional speed and efficiency.
- Direct hardware interaction.
- Real-time control capabilities.
- Extensive community support.

Cons:

- Steeper learning curve.
- More prone to errors.

Hardware compatibility:

- Wide range of microcontrollers and development boards, such as Arduino, STM32, and Raspberry Pi.

SPACE FACULTY
2NDSpace

VHDL and Verilog

VHDL and Verilog are specialized hardware description languages exclusively used for configuring FPGAs. These languages are not intended for general-purpose programming but rather for precise hardware control.

Pros:

- Specialized for hardware description.
- Allows precise control of FPGA logic.
- Ideal for digital design and FPGA configuration.

Cons:

- Not suitable for general-purpose programming.
- Requires specific knowledge of FPGA hardware.

Hardware compatibility:

- VHDL and Verilog are exclusively used for configuring Field-Programmable Gate Arrays (FPGAs).

SPACE FACULTY
2NDSpace

Introduction to Coding

Definition: Coding is the process of instructing a computer to perform a specific task by providing it with a set of step-by-step instructions.

SPRICE FACULTY
2NDSpace

Basic Syntax

A program consists of a series of instructions that the computer executes sequentially.

Essential components of a program:

- **Header Files**: Explain that header files like #include <stdio.h> are used to include libraries that provide essential functions and features.
- **main() Function**: Mention that every C program starts with the main() function, which serves as the entry point for the program.
- **Statements and Semicolons**: Emphasize that statements (lines of code) are terminated with semicolons (;) to indicate the end of an instruction.

SPRICE FACULTY
2NDSpace

Basic Syntax

```c
#include <stdio.h>

int main() {
    // This is a comment
    printf("Hello, World!"); // Print a message
    return 0; // Return 0 to indicate successful execution
}
```

SPACE FACULTY
2NDSpace

Variables and Data Types

Variables are “containers” for storing data in a program.

Data types:

- **int**: For integer values (e.g., 42)
- **float**: For floating-point numbers (e.g., 3.14)
- **char**: For single characters (e.g., 'A') Explain the importance of choosing the right data type for efficiency and correctness. Mention naming conventions for variables, such as starting with a letter and using underscores (e.g., my_variable).

SPACE FACULTY
2NDSpace

Variables and Data Types

Example:

```
int age = 25;
float temperature = 98.6;
char grade = 'A';
```

SPACE FACULTY
2NDSpace

Control Structures

Control structures allow you to make decisions and perform repetitive tasks in a program:

- **if-else statements**: Describe conditional statements and how they allow you to execute code based on conditions.

```
if (age >= 18) {
    printf("You are an adult.");
} else {
    printf("You are a minor.");
}
```

- Loops (**While** and **For**).

SPACE FACULTY
2NDSpace

Control Structures

- **While Loop**: Repeats a block of code as long as a specified condition is true.

```
while (condition) {
    // Code to repeat
}
```

- **For Loop**: Used when you know the number of iterations in advance.

```
for (initialization; condition; increment/decrement) {
    // Code to repeat
}
```

SPRICE FACULTY
2NDSpace

Functions

A function is a reusable block of code that performs a specific task.

Function components:

- Declaration
- Definition
- Parameters and Return Values

```
// Function declaration
int add(int a, int b);

// Function definition
int add(int a, int b) {
    return a + b;
}
```

SPACE FACULTY
2NDSpace

Functions

A function is a reusable block of code that performs a specific task.

Function components:

- Declaration
- Definition
- Parameters and Return Values

```
// Function declaration
int add(int a, int b);

// Function definition
int add(int a, int b) {
    return a + b;
}
```

SPACE FACULTY
2NDSpace

Functions

Declaration: tells the compiler about the function's name, return type, and the types of its parameters (but not their names). A function declaration is necessary when you want to use a function before its actual definition in your code.

```
int add(int a, int b);   // Function declaration
```

Definition: it includes the function's name, its return type, the parameter list (if any), and the code block that specifies what the function does. In a function definition, you write the actual code that gets executed when the function is called.

```
int add(int a, int b) {   // Function definition
    return a + b;
}
```

SPACE FACULTY
2NDSpace

Functions

Parameters: Parameters allow you to pass data into a function so that it can perform its task with specific values. In the function definition, you provide names for the parameters, and these names are used within the function's code block to work with the passed-in values.

```
int add(int a, int b) {   // 'a' and 'b' are parameters
    return a + b;
}
```

Return Values: values that a function sends back to the code that called it. When a function is called, it may perform some computations and return a result using the return statement. The return value can be of any data type, depending on the function's declaration.

```
int add(int a, int b) {   // 'int' indicates the return type
    return a + b;         // The result of 'a + b' is returned
}
```

SPACE FACULTY
2NDSpace

How to choose an OBC

CPU clock speed:

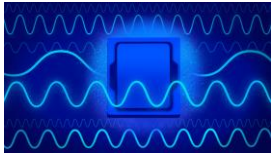
Suppose your CubeSat mission requires a specific data processing speed, such as processing sensor data at a rate of 500,000 data points per second. Let's assume each data point requires 50 clock cycles for processing. Here's how you can calculate the required CPU clock speed:

- Data Points Processed Per Second = 500,000
- Clock Cycles per Data Point = 50

The required CPU clock speed (in Hz) can be calculated as:

- Required CPU Clock Speed = (Data Points Processed Per Second) × (Clock Cycles per Data Point)
- Required CPU Clock Speed = 500,000 data points/second × 50 clock cycles/data point = 25,000,000 Hz = 25 MHz

So, in this example, you would need a CPU with a clock speed of at least 25 MHz to meet the data processing requirements of your mission.



SPACE FACULTY
2NDSpace

How to choose an OBC

RAM:
Let's assume your CubeSat mission needs to store a set of sensor measurements in RAM for real-time analysis. Each measurement set occupies 256 KB of RAM, and you want to store 20 sets simultaneously. Here's how you can calculate the required RAM size:

- Size of Each Measurement Set = 256 KB
- Number of Sets in RAM = 20

The required RAM size (in KB) can be calculated as:
- Required RAM Size = (Size of Each Measurement Set) × (Number of Sets in RAM)
- Required RAM Size = 256 KB/set × 20 sets = 5,120 KB ≈ 5 MB

So, in this example, you would need at least about 5 MB of RAM to store the required sensor measurements for real-time analysis.

How to choose an OBC

SSD Memory Size:

Suppose your CubeSat mission has the following parameters:

- Data Storage Rate: 16 MB/hour (data generated by environmental sensors and cameras).
- Mission Duration: 365 days (approximately 8,760 hours).
- Data Retention Factor: 1.5 (to account for redundancy and margin).

Required SSD Size (GB) = (Data Storage Rate × Mission Duration) / (1024 MB/GB) × Data Retention Factor

Step-by-step:

- Total Data = 16 MB/hour × 8,760 hours = 140,160 MB
- In GB = 140,160 MB / 1024 = 136.88 GB
- With Retention Factor = 136.88 × 1.5 = 205.32 GB

Required SSD Memory Size (GB) = 205.32 GB

So, in this example, you would need approximately 205 GB of SSD storage to safely accommodate the mission data with margin.

SPACE FACULTY
2NDSpace

How to choose an OBC

Power Consumption:
In CubeSat missions, the On-Board Computer is a key subsystem, but it can also be one of the most power-hungry ones.

Its consumption has a direct impact on the overall power budget, so efficient power management is essential. Selecting and configuring an OBC is always a trade-off between capability and available power.

In many missions, low power consumption becomes a major driver in the choice of the OBC - sometimes even more important than maximum CPU performance or memory size.

SPACE FACULTY
2NDSpace

How to choose an OBC

Interface Requirements:

UART (Universal Asynchronous Receiver-Transmitter):
- Needed for simple, robust point-to-point communication with sensors or subsystems for command and low/medium-rate data exchange.

I2C (Inter-Integrated Circuit):
- Needed for connecting multiple low-speed devices on a shared bus (e.g., environmental sensors, housekeeping components, small memories).

SPI (Serial Peripheral Interface):
- Needed for high-speed peripherals such as external memory, IMUs/ADCS modules, or fast sensors where bandwidth is critical.

GPIO (General-Purpose Input/Output):
- Needed for discrete digital signals such as enable/reset lines, interrupts, power-good/fault signals, and simple control commands.

Ethernet:
- Needed when the payload generates large volumes of data or requires network-based architectures (e.g., high-resolution cameras, onboard AI processors, payload OBCs).

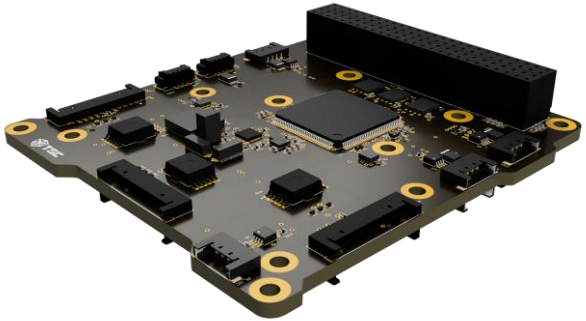
SpaceWire:
- Needed for high-speed, low-latency, space-qualified communication between the OBC and advanced payloads (e.g., hyperspectral imagers, SAR, complex scientific instruments).

SPACE FACULTY
2NDSpace

How to choose an OBC

Example configuration:

- Processor: e.g. ARM Cortex M4
- CPU Clock Speed: e.g. 100 MHz
- RAM Memory: e.g. 128 KB
- SSD Memory: 256GB
- Flash Memory: e.g. 256KB
- Power Consumption: e.g. 1W
- Interfaces: e.g. UART, I2C, SPI, GPIO



Radiation Vulnerability

Radiation in space can affect a Cubesat On-Board Computer and On-Board Data Handling system in several ways:

- Data Corruption: High-energy particles in space can strike OBC and OBDH components, causing transient errors or "bit flips" in memory and data registers. This radiation-induced interference can corrupt data, potentially leading to incorrect calculations or system malfunctions.
- Single-Event Effects (SEEs): Radiation can trigger SEEs, which are sudden, one-time disruptions in electronic circuits. This may result in temporary glitches or even permanent damage to components within the systems.
- Degrading Components: Over time, exposure to radiation can degrade the performance and reliability of OBC and OBDH components. This degradation may reduce the lifespan and effectiveness of the systems.

SPACE FACULTY
2NDSpace

Radiation Vulnerability: Workarounds

Through various strategies and technologies, engineers can ensure that Cubesats are capable of enduring and resisting radiation during space missions:

- Radiation-Hardened Components: Specially designed parts that resist radiation damage.
- Error-Detection Software: Intelligent software that spots and fixes radiation-induced errors.
- Redundancy and Backups: Spare systems that take over if radiation affects primary ones.
- Shielding and Armor: Protective layers that block harmful radiation.
- Rigorous Testing: Extensive simulations to prepare for radiation challenges.

SPACE FACULTY
2NDSpace

Innovative Technologies in OBCs

AI and Machine Learning:

- Onboard AI enables real-time data processing, anomaly detection, autonomous decision-making, and payload data filtering (e.g., selecting only relevant images to downlink). This reduces bandwidth usage and increases mission autonomy.

Swarm Intelligence:

- In multi-satellite missions, OBCs can implement distributed algorithms that allow CubeSats to coordinate autonomously, share data, and make collective decisions without constant ground intervention.

Quantum Computing (Experimental/Future Concept):

- Although not yet practical for CubeSat OBCs, quantum-inspired algorithms and future miniaturized quantum processors could enable advanced optimization, secure communications, or high-performance computing in space missions.

SPACE FACULTY
2NDSpace